A/B Tesing : Impact of Social Proof Badge on Food Delivery Orders

- Evaluate whether displaying a social-proof badge increases order conversion without negatively impacting order quality.

```
from google.colab import files
uploaded = files.upload()
```

Choose files ab_testingg…ys_fixed.csv

ab_testingg_14days_fixed.csv(text/csv) - 4743925 bytes, last modified: 03/04/2026 - 100% done

```
Saving ab_testingg_14days_fixed.csv to ab_testingg_14days_fixed.csv
```

```
import pandas as pd
import numpy as np

df = pd.read_csv("ab_testingg_14days_fixed.csv")
df.head()
```

	user_id	date	variant	ordered	menu_clicks	order_cancelled	time_to_order	order_value	user_type	day_of_week
0	D0_U00000	01-12-2025	control	0	6	0	NaN	NaN	new	6
1	D0_U00001	01-12-2025	control	0	8	0	NaN	NaN	returning	6
		01-								

Next steps: Generate code with df New interactive sheet

Data Quality & Sanity Checks

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 88708 entries, 0 to 88707
Data columns (total 10 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   user_id          88708 non-null  object
 1   date             88708 non-null  object
 2   variant          88708 non-null  object
 3   ordered          88708 non-null  int64
 4   menu_clicks      88708 non-null  int64
 5   order_cancelled  88708 non-null  int64
 6   time_to_order    18197 non-null  float64
 7   order_value      18197 non-null  float64
 8   user_type        88708 non-null  object
 9   day_of_week      88708 non-null  int64
dtypes: float64(2), int64(4), object(4)
memory usage: 6.8+ MB
```

```
df.isna().mean().sort_values(ascending=False)
```

	0
time_to_order	0.794866
order_value	0.794866
date	0.000000
user_id	0.000000
variant	0.000000
ordered	0.000000
order_cancelled	0.000000
menu_clicks	0.000000
user_type	0.000000
day_of_week	0.000000

dtype: float64

Sample Ratio Mismatch (SRM)

```
from scipy.stats import chisquare

counts = df['variant'].value_counts()
observed = counts.values
expected = [counts.sum()/2, counts.sum()/2]

chi2, p = chisquare(observed, expected)
chi2, p
```

```
(np.float64(0.28858727510483834), np.float64(0.5911272630262103))
```

Conversion Rate by Variant (Primary Metric)

```
conv = df.groupby('variant')['ordered'].mean()
conv
```

variant	ordered
control	0.201692
treatment	0.208587

dtype: float64

```
lift_pp = (conv['treatment'] - conv['control']) * 100
lift_rel = (conv['treatment'] / conv['control'] - 1) * 100
lift_pp, lift_rel
```

```
(np.float64(0.689503509134376), np.float64(3.418589480570944))
```

Hypothesis Testing (Primary Metric)

```
from statsmodels.stats.proportion import proportions_ztest

summary = df.groupby('variant')['ordered'].agg(['sum','count'])

z_stat, p_value = proportions_ztest(
    count=[summary.loc['treatment','sum'],
           summary.loc['control','sum']],
    nobs=[summary.loc['treatment','count'],
          summary.loc['control','count']],
    alternative='larger'
)

z_stat, p_value
```

```
(np.float64(2.542855173154399), np.float64(0.005497539597022649))
```

```
from statsmodels.stats.proportion import proportion_confint

# CI for each group's conversion rate
ci_control = proportion_confint(summary.loc['control','sum'], summary.loc['control','count'], alpha=0.05, method='normal')
```

```
ci_treatment = proportion_confint(summary.loc['treatment','sum'], summary.loc['treatment','count'], alpha=0.05, method='nor

# CI on the lift itself
se_diff = np.sqrt(
    conv['control']*(1-conv['control'])/summary.loc['control','count'] +
    conv['treatment']*(1-conv['treatment'])/summary.loc['treatment','count']
)
lift_ci_lo = (conv['treatment'] - conv['control']) - 1.96*se_diff
lift_ci_hi = (conv['treatment'] - conv['control']) + 1.96*se_diff

print(f"Control:   {conv['control']:.3f}  95% CI [{ci_control[0]:.3f}, {ci_control[1]:.3f}]")
print(f"Treatment: {conv['treatment']:.3f}  95% CI [{ci_treatment[0]:.3f}, {ci_treatment[1]:.3f}]")
print(f"Lift CI:   [{lift_ci_lo*100:.2f}pp, {lift_ci_hi*100:.2f}pp]")
```

```
Control:   0.202  95% CI [0.198, 0.205]
Treatment: 0.209  95% CI [0.205, 0.212]
Lift CI:   [0.16pp, 1.22pp]
```

Guardrail Metric — Cancellation Rate

```
cancel = df[df['ordered']==1].groupby('variant')['order_cancelled'].mean()
cancel
```

variant	order_cancelled
control	0.057242
treatment	0.063021

dtype: float64

```
cancel_df = df[df['ordered']==1]
summary_cancel = cancel_df.groupby('variant')['order_cancelled'].agg(['sum','count'])

z_cancel, p_cancel = proportions_ztest(
    count=[summary_cancel.loc['treatment','sum'],
           summary_cancel.loc['control','sum']],
    nobs=[summary_cancel.loc['treatment','count'],
          summary_cancel.loc['control','count']],
    alternative='two-sided'
)

z_cancel, p_cancel
```

```
(np.float64(1.6389857557673162), np.float64(0.10121622627997753))
```

Secondary Metric — Order Value

```
df[df['ordered']==1].groupby('variant')['order_value'].mean()
```

variant	order_value
control	321.651248
treatment	345.577496

dtype: float64

```
df[df['ordered']==1].groupby('variant')['time_to_order'].mean()
```

variant	time_to_order
control	12.95551
treatment	12.02006

dtype: float64

```
df.groupby('variant')['menu_clicks'].mean()
```

variant	menu_clicks
control	7.642886
treatment	8.586552

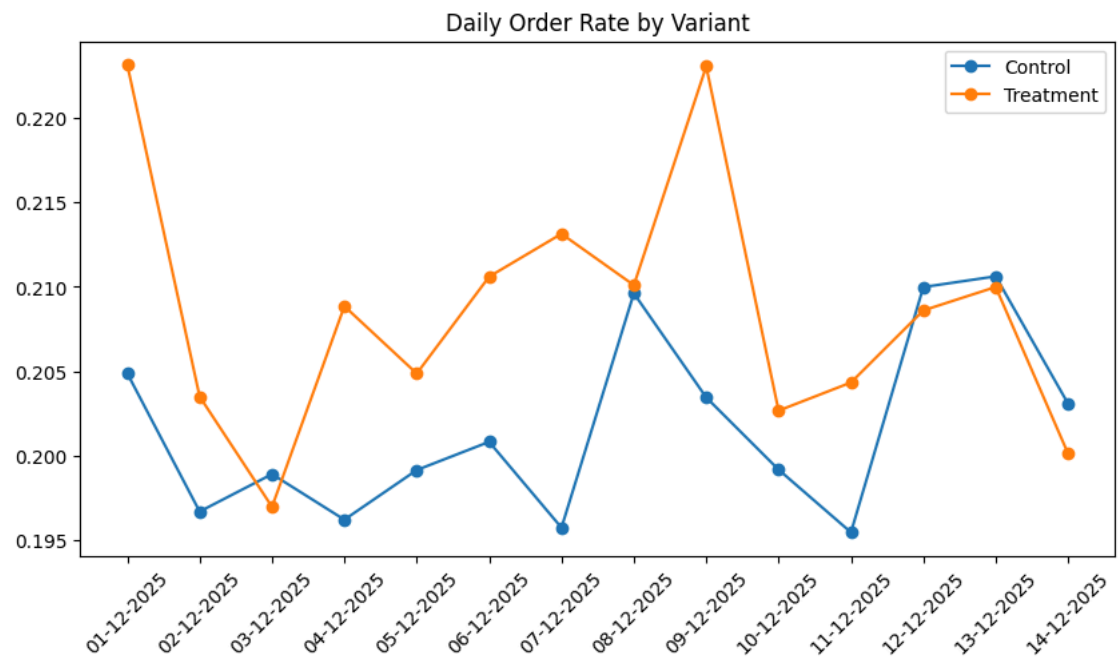
dtype: float64

```
from scipy.stats import ttest_ind

# Order value t-test
ctrl_vals  = df[(df['ordered']==1) & (df['variant']=='control')]['order_value'].dropna()
treat_vals = df[(df['ordered']==1) & (df['variant']=='treatment')]['order_value'].dropna()

t_stat, p_val = ttest_ind(treat_vals, ctrl_vals, alternative='two-sided')
print(f"Order value — Control: {ctrl_vals.mean():.1f}  Treatment: {treat_vals.mean():.1f}")
print(f"t={t_stat:.3f}, p={p_val:.4f}  {'SIGNIFICANT' if p_val < 0.05 else 'not significant'}")

# Time to order t-test (same pattern)
ctrl_time  = df[(df['ordered']==1) & (df['variant']=='control')]['time_to_order'].dropna()
treat_time = df[(df['ordered']==1) & (df['variant']=='treatment')]['time_to_order'].dropna()

t2, p2 = ttest_ind(treat_time, ctrl_time, alternative='two-sided')
print(f"\nTime to order — Control: {ctrl_time.mean():.1f}  Treatment: {treat_time.mean():.1f}")
print(f"t={t2:.3f}, p={p2:.4f}  {'SIGNIFICANT' if p2 < 0.05 else 'not significant'}")
```

```
Order value — Control: 321.7  Treatment: 345.6
t=14.929, p=0.0000  SIGNIFICANT

Time to order — Control: 13.0  Treatment: 12.0
t=-12.083, p=0.0000  SIGNIFICANT
```

EDA

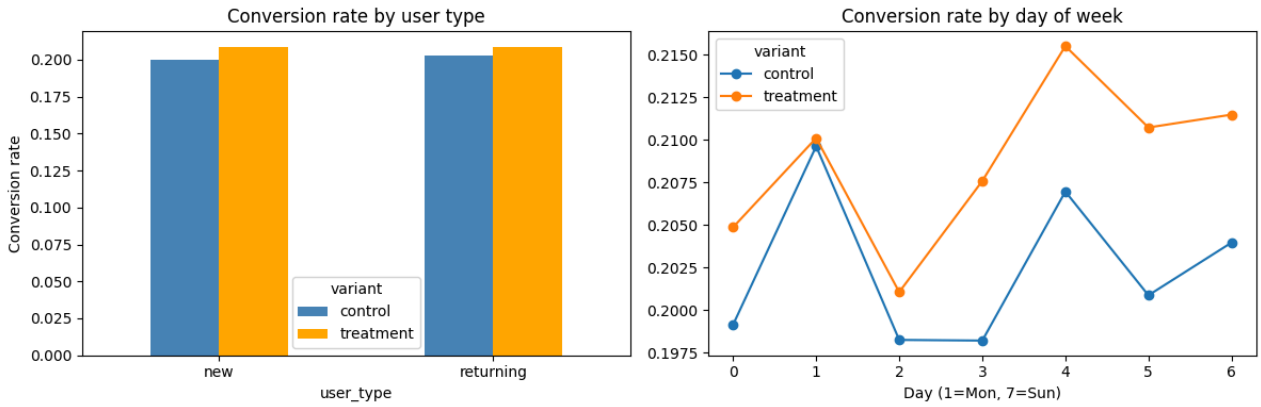
```
daily = df.groupby(['date','variant'])['ordered'].mean().unstack()

import matplotlib.pyplot as plt
plt.figure(figsize=(10,5))
plt.plot(daily.index, daily['control'], marker='o', label='Control')
plt.plot(daily.index, daily['treatment'], marker='o', label='Treatment')
plt.legend()
plt.xticks(rotation=45)
plt.title("Daily Order Rate by Variant")
plt.show()
```




```
# New vs Returning users
seg = df.groupby(['variant','user_type'])['ordered'].agg(['mean','count']).unstack('variant')
print("Conversion rate by user type:")
print(seg['mean'].round(4))

# Day of week effect
```

```
dow_seg = df.groupby(['variant','day_of_week'])['ordered'].mean().unstack('variant')
dow_seg['lift_pp'] = (dow_seg['treatment'] - dow_seg['control']) * 100
print("\nLift by day of week:")
print(dow_seg.round(4))

# Visualise segmentation
import matplotlib.pyplot as plt

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# User type bars
user_conv = df.groupby(['user_type','variant'])['ordered'].mean().unstack()
user_conv.plot(kind='bar', ax=axes[0], color=['steelblue','orange'])
axes[0].set_title("Conversion rate by user type")
axes[0].set_ylabel("Conversion rate")
axes[0].set_xticklabels(axes[0].get_xticklabels(), rotation=0)

# Day of week line
dow_seg[['control','treatment']].plot(ax=axes[1], marker='o')
axes[1].set_title("Conversion rate by day of week")
axes[1].set_xlabel("Day (1=Mon, 7=Sun)")

plt.tight_layout()
plt.show()
```

```
Conversion rate by user type:
variant    control  treatment
user_type
new         0.2002     0.2085
returning   0.2027     0.2087

Lift by day of week:
variant      control  treatment  lift_pp
day_of_week
0             0.1992     0.2049   0.5716
1             0.2096     0.2101   0.0482
2             0.1983     0.2011   0.2808
3             0.1982     0.2076   0.9364
4             0.2070     0.2155   0.8533
5             0.2009     0.2107   0.9845
6             0.2040     0.2115   0.7510
```




```
# ── Novelty check — Week 1 vs Week 2 ──────────────────────────
df['date_parsed'] = pd.to_datetime(df['date'], dayfirst=True)
df['week'] = df['date_parsed'].apply(lambda x: 'week_1' if x.day <= 7 else 'week_2')
novelty = df.groupby(['week','variant'])['ordered'].mean().unstack()
novelty['lift_pp'] = (novelty['treatment'] - novelty['control']) * 100
w1 = novelty.loc['week_1','lift_pp']
w2 = novelty.loc['week_2','lift_pp']
print(novelty.round(4))
print(f"\nWeek 1 lift: +{w1:.2f}pp | Week 2 lift: +{w2:.2f}pp | Durability: {w2/w1*100:.0f}%")
print("Novelty suspected — lift decayed in week 2. Monitor conversion for 4 weeks post-ship.")
```

```
variant  control  treatment  lift_pp
week
week_1    0.1989     0.2088   0.9879
week_2    0.2045     0.2084   0.3879

Week 1 lift: +0.99pp | Week 2 lift: +0.39pp | Durability: 39%
Novelty suspected — lift decayed in week 2. Monitor conversion for 4 weeks post-ship.
```